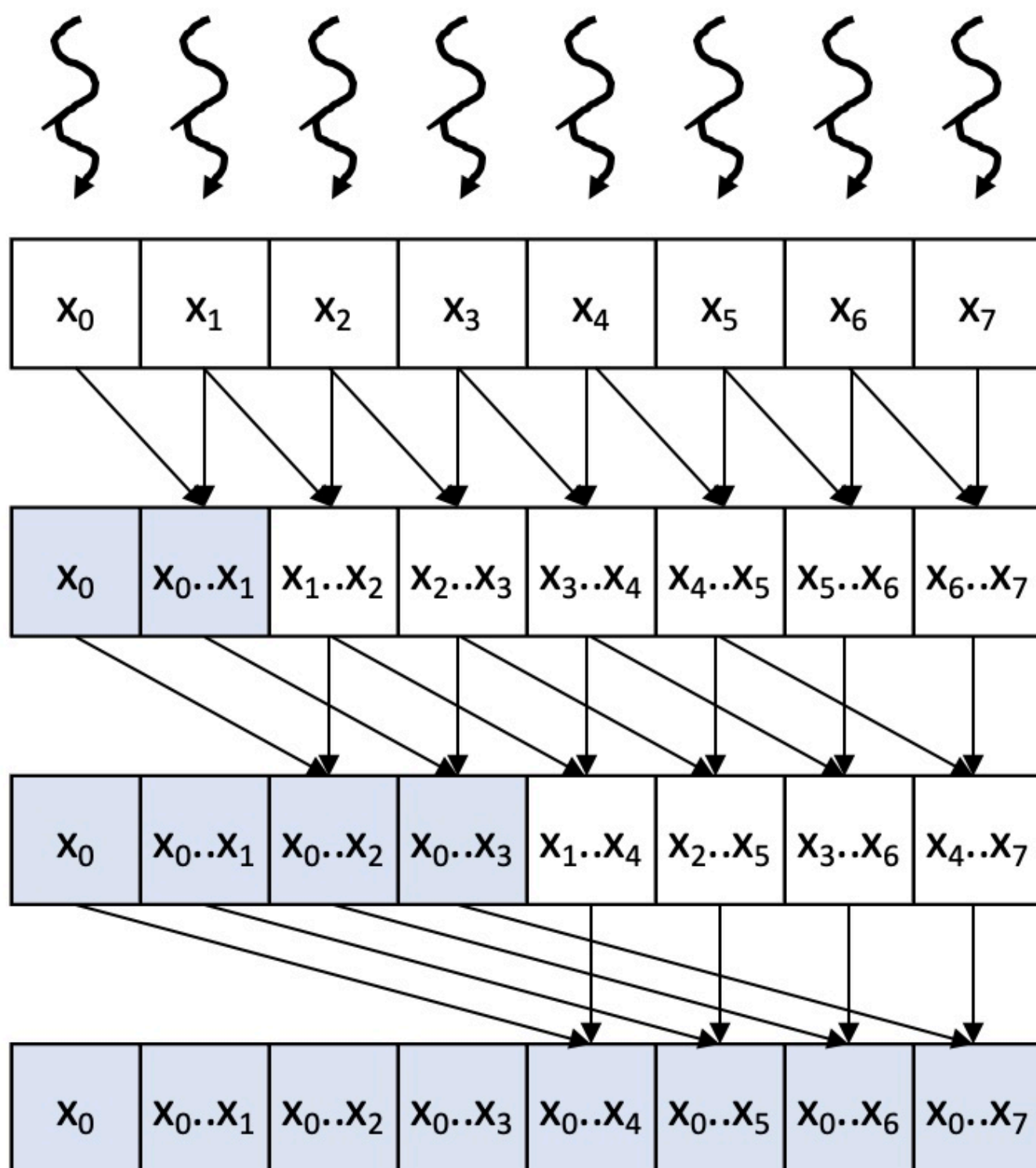


Kogge-Stone



Double Buffering

```
unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;

__shared__ float buffer1_s[BLOCK_DIM];
__shared__ float buffer2_s[BLOCK_DIM];
float* inBuffer_s = buffer1_s;
float* outBuffer_s = buffer2_s;
inBuffer_s[threadIdx.x] = input[i];
__syncthreads();

for(unsigned int stride = 1; stride <= BLOCK_DIM/2; stride *= 2) {
    if(threadIdx.x >= stride) {
        outBuffer_s[threadIdx.x] =
            inBuffer_s[threadIdx.x] + inBuffer_s[threadIdx.x - stride];
    } else {
        outBuffer_s[threadIdx.x] = inBuffer_s[threadIdx.x];
    }
    __syncthreads();
    float* tmp = inBuffer_s;
    inBuffer_s = outBuffer_s;
    outBuffer_s = tmp;
}

if(threadIdx.x == BLOCK_DIM - 1) {
```

```
partialSums[blockIdx.x] = inBuffer_s[threadIdx.x];
```

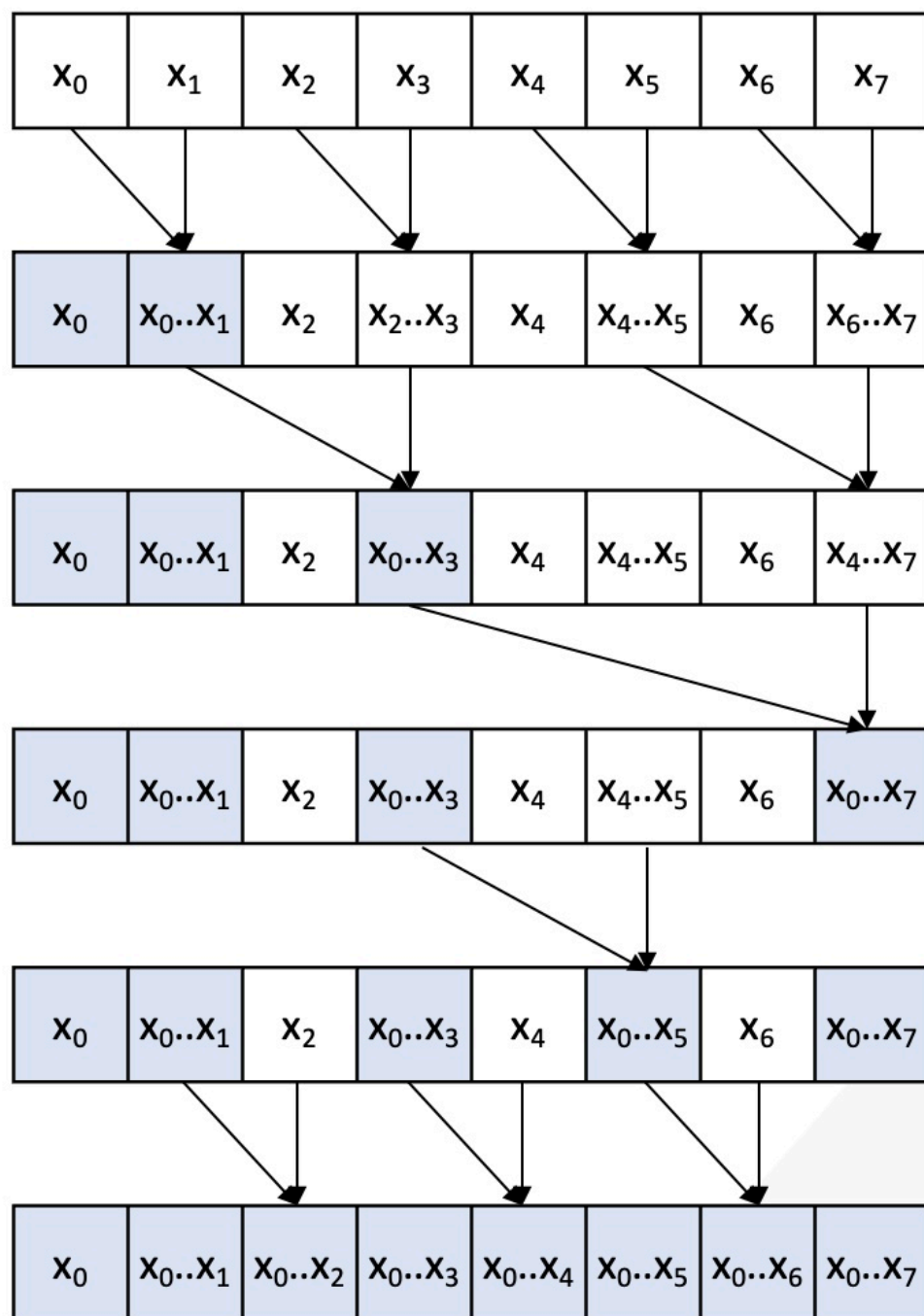
```
}
```

```
output[i] = inBuffer_s[threadIdx.x];
```

Control Divergence Analysis:

- During iteration 1–5 (stride: 1, 2, 4, 8, 16), the first warp will have control divergence only.
- For the remaining iterations (stride: 32, 64, ...), entire warps will deactivate & as such there will be no control divergence.

Brent-Kung



1. State After the Reduction Phase

The top half of Figure 11.5 represents a reduction tree. After this initial phase completes, only specific positions in the array (indices that are powers of 2 minus 1: 0, 1, 3, 7, 15) contain their final, complete inclusive scan results. All other positions contain only "partial sums," representing a restricted continuous range of the original input elements (expressed as $x_i \dots x_j$).

2. Constructing the Missing Values

To complete the scan, the positions that only have partial sums must gather additional values to form their final complete sum. The text uses index 14 (`XY[14]`) as a detailed example:

- At the end of the reduction phase, `XY[14]` only contains the value of x_{14} .
- To possess the final inclusive scan value, it must contain the sum of all elements from x_0 through x_{14} .
- It achieves this by accumulating previously computed partial sums from earlier positions in the array:
 - `XY[7]` provides the accumulated sum of $x_0 \dots x_7$.
 - `XY[11]` provides the accumulated sum of $x_8 \dots x_{11}$.
 - `XY[13]` provides the accumulated sum of $x_{12} \dots x_{13}$.
- Adding these specific partial sums to the existing x_{14} yields the complete sequence sum $x_0 \dots x_{14}$.

3. The Mathematical Upper Bound (Highlighted Text)

The highlighted section establishes a definitive limit on the maximum number of extra partial sums any single position must fetch during the reverse tree phase.

- **Formula:** For an input array of size N , a position will never need to accumulate more than $\log_2(N) - 1$ partial sums from other positions.
- **Example Verification:** For an array of $N = 16$ elements, the maximum required additions is $\log_2(16) - 1 = 4 - 1 = 3$. The `XY[14]` example demonstrates this theoretical maximum, requiring exactly 3 additional partial sums to complete its value.
- **Spacing:** The text also notes a structural guarantee: these required partial sums will always be located at index distances that are powers of 2 (1, 2, 4...) away from each other in the array.

- **Level 1 (Distance of 4):** The operation begins with the largest required gap. The partial sum at `XY[7]` is added to `XY[11]`. This single operation completes the final inclusive sum for position 11.
- **Level 2 (Distance of 2):** The distance halves. Elements now receive partial sums from exactly two positions to their left. `XY[3]` is added to `XY[5]`, `XY[7]` is added to `XY[9]`, and the newly completed `XY[11]` is added to `XY[13]`. This finishes the calculations for positions 5, 9, and 13.
- **Level 3 (Distance of 1):** The distance halves again. The remaining unresolved even positions (`XY[2]`, `XY[4]`, `XY[6]`, `XY[8]`, `XY[10]`, `XY[12]`) reach their final state by accumulating the value from their immediate left neighbor (a distance of 1).

The paragraph demonstrates that the reverse tree resolves missing values systematically by

starting with the widest strides and halving the distance at each subsequent level until all positions in the array contain their final scanned values.

Reduction step:

```
for(unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {  
    __syncthreads();  
    if ((threadIdx.x + 1) % (2 * stride) == 0) {  
        XY[threadIdx.x] += XY[threadIdx.x - stride];  
    }  
}
```

→ This is making the even threads have odd id & odd threads have even id.

By modulo by $2 * \text{stride}$, we are only letting threads with form $2^n - 1$

- This kernel has significant control divergence. So, we re-index thread → data every iteration.

```
for(unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {  
    __syncthreads();  
    int index = (threadIdx.x + 1) * 2 * stride - 1;  
    if(index < SECTION_SIZE) {  
        XY[index] += XY[index - stride];  
    }  
}
```

$$\begin{aligned} \text{tx: } 0, \text{ stride} &= 1 \\ \text{id}x &= 2(0+1) - 1 \\ &= 1 \end{aligned}$$

$$XY[1] = XY[1] + XY[0]$$

$$\begin{aligned} \text{tx: } 0, \text{ stride} &= 2 \\ \text{id}x &= 4(0+1) - 1 \\ &= 3 \end{aligned}$$

$$XY[3] = XY[3] + XY[1]$$

Reverse Reduction Step:

- Each thread adds its assigned element to + stride location.

```
for (int stride = SECTION_SIZE/4; stride > 0; stride /= 2) {  
    __syncthreads();  
    int index = (threadIdx.x + 1) * stride * 2 - 1;  
    if(index + stride < SECTION_SIZE) {  
        XY[index + stride] += XY[index];  
    }  
}
```

$$tx: 0, stride = 16/4 = 4$$

$$idx = 8(0+1) - 1 = 7$$

$$XY[11] = XY[11] + XY[7]$$

$$tx: 0, stride = 2$$

$$idx = 4(0+1) - 1 = 3$$

$$XY[5] = XY[5] + XY[3]$$

```

01  __global__ void Brent_Kung_scan_kernel(float *X, float *Y, unsigned int N) {
02  __shared__ float XY[SECTION_SIZE]; } → section-size = 2 * blockDim.x
03  unsigned int i = 2*blockIdx.x*blockDim.x + threadIdx.x;
04  if(i < N) XY[threadIdx.x] = X[i];
05  if(i + blockDim.x < N) XY[threadIdx.x + blockDim.x] = X[i + blockDim.x];
06  for(unsigned int stride = 1; stride <= blockDim.x; stride *= 2) {
07      __syncthreads();
08      unsigned int index = (threadIdx.x + 1)*2*stride - 1;
09      if(index < SECTION_SIZE) {
10          XY[index] += XY[index - stride];
11      }
12  }
13  for (int stride = SECTION_SIZE/4; stride > 0; stride /= 2) {
14      __syncthreads();
15      unsigned int index = (threadIdx.x + 1)*stride*2 - 1;
16      if(index + stride < SECTION_SIZE) {
17          XY[index + stride] += XY[index];
18      }
19  }
20  __syncthreads();
21  if (i < N) Y[i] = XY[threadIdx.x];
22  if (i + blockDim.x < N) Y[i + blockDim.x] = XY[threadIdx.x + blockDim.x];
23  }

```

→ Load/store from/in global memory in a coalesced manner

Work efficiency of Brent-Kung

1. Reverse Tree Operations (Example $N = 16$)

The text begins by calculating the operations in the reverse tree phase for an array of 16 elements. The number of addition operations at each level follows the pattern $(16/x) - 1$:

- **Level 1:** $(16/8) - 1 = 1$ operation.
- **Level 2:** $(16/4) - 1 = 3$ operations.
- **Level 3:** $(16/2) - 1 = 7$ operations.

2. Generalizing the Reverse Tree Formula

For any input size N , the number of operations per level forms a series: $(2 - 1) + (4 - 1) + \dots + (N/2 - 1)$.

Summing this algebraic sequence yields the total number of operations strictly for the reverse tree phase:

$$N - 1 - \log_2(N)$$

3. Total Operations for the Entire Algorithm

The complete Brent-Kung scan algorithm consists of two sequential phases. To find the overall operation count, the sums from both phases are combined:

- **Reduction Tree Phase:** $N - 1$ operations.
- **Reverse Tree Phase:** $N - 1 - \log_2(N)$ operations.
- **Total Operations:** $(N - 1) + (N - 1 - \log_2(N)) = 2N - 2 - \log_2(N)$.

4. Algorithmic Complexity (Big-O Notation)

- **Brent-Kung:** As N becomes very large, the linear term $2N$ dominates the formula, making the logarithmic term and constants insignificant. Dropping the constant coefficient results in an asymptotic work complexity of $O(N)$.
- **Kogge-Stone:** The text contrasts this with the Kogge-Stone algorithm, which has a complexity of $O(N \log_2 N)$.
- **Conclusion:** Because $O(N)$ scales significantly better than $O(N \log_2 N)$, the Brent-Kung algorithm is mathematically proven to be more work-efficient, performing far fewer total operations as the dataset size increases.

Control Divergence Analysis

Reduction Phase (Forward Tree)

- **No Divergence Early:** The loop starts with all threads active. As the `stride` doubles, the number of active threads halves. Because active threads remain contiguous at the lowest indices, warps are either entirely active or entirely inactive.
- **Divergence Late:** Control divergence only occurs during the final five iterations. In these steps, the total number of active threads drops below 32 (16, 8, 4, 2, and 1). This causes the first warp to partially execute the `if (index < SECTION_SIZE)` block, resulting in intra-warp divergence.

Distribution Phase (Reverse Tree)

- **Divergence Early:** This phase begins with the maximum stride and only 1 active thread. Divergence occurs during the first five iterations as the active thread count doubles (1, 2, 4, 8, 16), causing partial execution within the first warp.
- **No Divergence Late:** Once the number of active threads reaches 32, the first warp is fully active. In all subsequent iterations, active threads increase in multiples of 32. Entire warps either fully pass or fully fail the `if (index + stride < SECTION_SIZE)` condition, resulting in zero intra-warp divergence.

The main disadvantage of Brent-Kung over Kogge-Stone is its potentially longer execution time despite its higher work efficiency. With infinite execution resources, Brent-Kung should take about twice as long as Kogge-Stone, owing to the need for additional steps to perform the reverse tree phase. However, the speed comparison can be quite different when we have limited execution resources. Using the example in Section 11.3, if we process 1024 input elements with 32 execution units, the Brent-Kung kernel is expected to take approximately $(2 \cdot 1024 - 2 - 10) / 32 = 63.6$ steps. The reader should verify that with control divergence there will be about five more steps when the total number of active threads drops below 32 in each phase. This results in a speedup of $1024 / 73.6 = 14$ over sequential execution. This is in comparison with the 320 time units and speedup of 3.2 for Kogge-Stone. The comparison would be to Kogge-Stone's advantage, of course, when there are many more execution resources and/or when the latencies are much longer.

- **Theoretical vs. Actual:** The theoretical calculation of 63.6 steps assumes 100% continuous utilization of the 32 execution units.
- **The SIMD Penalty:** In both the reduction phase and the reverse tree phase, there are exactly 5 iterations where the number of active threads drops below the warp size of 32 (iterations requiring 16, 8, 4, 2, and 1 active threads).
- **The Math:** Theoretically, computing these 31 operations with 32 execution units should take roughly 1 execution step ($31 / 32$). In physical hardware, because threads execute in SIMD warps, the GPU must consume a full execution step for each of those 5 separate iterations. This introduces a penalty of approximately 5 execution steps per phase.
- **Final Calculation:** Adding 5 extra steps for the reduction phase and 5 extra steps for the reverse phase to the theoretical 63.6 steps yields the final hardware execution time of 73.6 steps.

more clarification

- **The Theoretical Flaw:** The mathematical formula simply counts the total operations across those 5 iterations ($16 + 8 + 4 + 2 + 1 = 31$ operations). It assumes that because 31 is less than 32, the hardware can do all of this work simultaneously in 1 step.
- **The Penalty:** Because the GPU must actually process these operations across 5 distinct, sequential loop iterations, it dispatches Warp 0 five separate times. This takes 5 execution steps instead of the theoretically assumed 1 step.

Work Efficiency of Thread Coarsening

- Phase 1 (Sequential Scan within Chunks):** Each of the T threads performs a sequential scan on its assigned chunk. Since each chunk has N/T elements, it takes $(N/T) - 1$ operations per chunk. Across all T threads, the total work is $T \cdot ((N/T) - 1) = N - T$ operations.
- Phase 2 (Parallel Scan across Chunks):** The algorithm takes the final sum from each of the T chunks and runs a standard Kogge-Stone parallel scan on just those T values. As established previously, Kogge-Stone requires $T \log_2(T)$ operations.
- Phase 3 (Distribution):** The results from Phase 2 are added back to the individual elements within each chunk. Each thread adds a single value to all but the last element in its chunk. This requires $(N/T) - 1$ additions per chunk. Across all T threads, the total work is again $T \cdot ((N/T) - 1) = N - T$ operations.

Total Work and Execution Steps

The total amount of work (operations) is the sum of the three phases:

$$\text{Total Work} = (N - T) + T \log_2(T) + (N - T)$$

To find the expected number of execution steps, the total work is divided by the number of physical execution units available (P):

$$\text{Execution Steps} = \frac{(N - T) + T \log_2(T) + (N - T)}{P}$$

The Example Calculation

The text provides an example to demonstrate the formula:

- N (elements): 1024
- T (threads): 64
- P (execution units): 32

Plugging these values into the formula:

$$\text{Steps} = \frac{(1024 - 64) + 64 \cdot \log_2(64) + (1024 - 64)}{32}$$

$$\text{Steps} = \frac{960 + 64 \cdot 6 + 960}{32}$$

$$\text{Steps} = \frac{960 + 384 + 960}{32} = \frac{2304}{32} = 72 \text{ steps}$$

